

Sistemas de Numeração e Representação de Dados

Resumo de estudo — Bits, armazenamento, bases numéricas e conversões

■ **Objetivo do resumo** — Compreender a representação de dados em nível de hardware (bits e bytes), entender por que o computador trabalha em binário, dominar os sistemas de numeração posicional (binário, octal, hexadecimal, BCD) e as técnicas de conversão entre bases.

Seção 1 — Conceitos Básicos: Bits

■ **Bit (Binary Digit)** — A menor unidade de informação em um computador. Assume apenas dois valores: **0** ou **1**. Toda informação digital — texto, imagem, som, vídeo — é, em última instância, uma sequência de bits.

1. Por que o Computador Não Trabalha em Decimal?

Os circuitos eletrônicos do computador operam com dois estados elétricos (ligado/desligado, alta/baixa tensão). Representar 10 estados distintos (0 a 9) exigiria circuitos muito mais complexos e sujeitos a erros. O sistema binário (2 estados) é naturalmente compatível com a eletrônica digital, tornando o hardware mais simples, confiável e rápido.

2. Armazenamento e Unidades de Memória

■ **Memória** — Organizada como um total de 2^n células endereçáveis. Cada célula armazena tipicamente 1 byte (8 bits).

Unidade	Símbolo	Equivalência	Potência de 2
Byte	B	8 bits	2^0 bytes = 1 B
Kilobyte	KB	1.024 bytes	2^{10} bytes
Megabyte	MB	1.024 KB	2^{20} bytes
Gigabyte	GB	1.024 MB	2^{30} bytes
Terabyte	TB	1.024 GB	2^{40} bytes

■ **Atenção** — Na computação, 1 KB = 1.024 bytes (2^{10}), não 1.000. Isso porque as memórias são dimensionadas em potências de 2.

Seção 2 — Sistemas de Numeração Posicional

1. Conceitos Fundamentais

Número — conceito abstrato de quantidade. **Numeral** — representação escrita de um número.

Algarismo — cada símbolo individual usado no numeral.

■ **Sistema posicional** — O valor de cada algarismo depende de sua **posição**. Em uma base b , um numeral com n dígitos inteiros e k dígitos fracionários representa:

$$N = a_{n-1} \cdot b^{n-1} + \dots + a_1 \cdot b^1 + a_0 \cdot b^0 + a_{-1} \cdot b^{-1} + \dots$$

- Exemplo (base 10): $2105 = 2 \cdot 10^3 + 1 \cdot 10^2 + 0 \cdot 10^1 + 5 \cdot 10^0$
- Exemplo (base 10, com fração): $12,45333... = 1 \cdot 10^1 + 2 \cdot 10^0 + 4 \cdot 10^{-1} + ...$

2. Visão Geral das Bases

Base	Nome	Algarismos	Bits/dígito	Uso principal
2	Binário	0, 1	1	Representação interna do computador
8	Octal	0 a 7	3	Permissões Unix; notação compacta
10	Decimal	0 a 9	~3,32	Sistema humano do dia a dia
16	Hexadecimal	0-9, A-F	4	Endereços de memória; cores (HTML/CSS)

Seção 3 — Sistema Binário (Base 2)

1. Conversão Binário → Decimal

Multiplique cada dígito pela potência de 2 correspondente à sua posição e some tudo.

■ Exemplo (inteiro):

$$(1101001)_2 = 1 \cdot 2^0 + 0 \cdot 2^1 + 0 \cdot 2^2 + 1 \cdot 2^3 + 0 \cdot 2^4 + 1 \cdot 2^5 + 1 \cdot 2^6 = 1 + 8 + 32 + 64 = 105$$

■ Exemplo (fracionário):

$$(110,1001)_2 = 1 \cdot 2^{-4} + 1 \cdot 2^{-1} + 1 \cdot 2^1 + 1 \cdot 2^2 = 1/16 + 0,5 + 2 + 4 = 6,5625$$

2. Conversão Decimal → Binário (Divisões Sucessivas)

■ Método — Divida o número inteiro por 2 sucessivamente, anotando os restos. A leitura dos restos de trás para frente forma o numeral binário.

■ Exemplo: $(731)_{10} \rightarrow (?)_2$

n	731	365	182	91	45	22	11	5	2	1	0
q = n/2	365	182	91	45	22	11	5	2	1	0	—
r (resto)	1	1	0	1	1	0	1	1	0	1	—

Lendo os restos de trás para frente: $(731)_{10} = (1011011011)_2$

■ Exemplo: $(513)_{10} \rightarrow (?)_2$

n	513	256	128	64	32	16	8	4	2	1	0
q = n/2	256	128	64	32	16	8	4	2	1	0	—
r (resto)	1	0	0	0	0	0	0	0	0	1	—

Lendo os restos de trás para frente: $(513)_{10} = (1000000001)_2$

Seção 4 — Sistema Octal (Base 8)

■ **Octal** — Usa algarismos de 0 a 7. Cada dígito octal equivale a exatamente **3 bits** (pois $2^3 = 8$). As potências da base são: $8^0 = 1$, $8^1 = 8$, $8^2 = 64$, $8^3 = 512$, ...

1. Conversão Octal → Decimal

Multiplique cada dígito pela potência de 8 correspondente e some.

■ **Exemplo:**

$$(135)_8 = 1 \cdot 8^2 + 3 \cdot 8^1 + 5 \cdot 8^0 = 64 + 24 + 5 = 93_{10}$$

2. Conversão Decimal → Octal (Divisões Sucessivas)

Mesmo método do binário, mas dividindo por 8.

■ **Exemplo:** $(157)_{10} \rightarrow (?)_8$

$157 \div 8 = 19$ (resto **5**) $\rightarrow 19 \div 8 = 2$ (resto **3**) $\rightarrow 2 \div 8 = 0$ (resto **2**)

Lendo de trás para frente: $(157)_{10} = (235)_8$

Seção 5 — Sistema Hexadecimal (Base 16)

■ **Hexadecimal** — Usa algarismos de 0 a 9 e letras A a F (onde A=10, B=11, C=12, D=13, E=14, F=15). Cada dígito hexadecimal equivale a exatamente **4 bits** (pois $2^4 = 16$).

1. Tabela de Correspondência Hex ↔ Decimal ↔ Binário

Hex	Dec	Bin		Hex	Dec	Bin
0	0	0000		8	8	1000
1	1	0001		9	9	1001
2	2	0010		A	10	1010
3	3	0011		B	11	1011
4	4	0100		C	12	1100
5	5	0101		D	13	1101
6	6	0110		E	14	1110
7	7	0111		F	15	1111

2. Conversão Hexadecimal → Decimal

■ **Exemplo:**

$$(2A6)_{16} = 2 \cdot 16^2 + 10 \cdot 16^1 + 6 \cdot 16^0 = 512 + 160 + 6 = 678_{10}$$

3. Conversão Decimal → Hexadecimal

■ **Exemplo:** $(250)_{10} \rightarrow (?)_{16}$

$250 \div 16 = 15$ (resto **10 = A**) $\rightarrow 15 \div 16 = 0$ (resto **15 = F**)

Lendo de trás para frente: $(250)_{10} = (FA)_{16}$

Seção 6 — Código BCD (Binary-Coded Decimal)

■ **BCD** — Grupo de símbolos codificados. Cada **dígito decimal** de um número é substituído individualmente pelo seu equivalente binário de **4 bits**. Diferente da conversão direta para binário!

■ **Exemplo: $(459)_{10}$ em BCD**

Dígito decimal	4	5	9
BCD (4 bits)	0100	0101	1001

Resultado: $(459)_{10} = 0100\ 0101\ 1001$ (BCD)

■ **BCD vs. Binário puro** — No binário puro, $(459)_{10} = (111001011)_2$ (9 bits). Em BCD, são 12 bits (4 por dígito $\times$ 3 dígitos). O BCD é menos eficiente em armazenamento, mas facilita a conversão direta para displays decimais (calculadoras, relógios digitais).

Síntese — Mapa do Resumo

Seção	Tema	Conceito-chave
1	Bits e Armazenamento	Bit = 0 ou 1; KB = 2^{10} bytes
2	Numeração Posicional	Valor = posição $\times$ potência da base
3	Sistema Binário	Divisões sucessivas por 2; leitura dos restos
4	Sistema Octal	1 dígito octal = 3 bits
5	Sistema Hexadecimal	1 dígito hex = 4 bits; A=10 ... F=15
6	Código BCD	Cada dígito decimal $\rightarrow$ 4 bits (codificação individual)

Resumo Rápido — Cola de Estudo

■ **Checklist** — • Explicar por que o computador usa base 2 (dois estados elétricos). • Converter KB, MB, GB usando potências de 2. • Definir sistema posicional e expandir um numeral em qualquer base. • Converter binário $\rightarrow$ decimal (somar potências de 2). • Converter decimal $\rightarrow$ binário (divisões sucessivas por 2). • Converter números fracionários entre bases. • Converter octal $\leftrightarrow$ decimal (potências de 8 / divisões por 8). • Converter hexadecimal $\leftrightarrow$ decimal (potências de 16 / divisões por 16). • Saber a tabela hex (A=10 ... F=15) de memória. • Codificar um número decimal em BCD (4 bits por dígito). • Distinguir BCD de conversão binária pura.

■ **Regras de ouro** — 1) Octal $\rightarrow$ binário: substitua cada dígito por 3 bits. Hex $\rightarrow$ binário: substitua cada dígito por 4 bits. 2) Divisões sucessivas: os restos são lidos *de trás para frente*. 3) BCD não é o mesmo que binário! $(459)_{10}$ em BCD $\neq$ $(459)_{10}$ em binário. 4) 1 KB = 1.024 bytes, não 1.000 — sempre potências de 2 em computação.

■ **Conexão com outras áreas** — Endereços de memória e MAC são escritos em hexadecimal. Permissões de arquivos no Unix usam octal (chmod 755). Cores em HTML/CSS usam hex (#FF0000 = vermelho). O entendimento de bases numéricas é essencial para depuração, programação de baixo nível e arquitetura de computadores.

#arquitetura #bits #binario #octal #hexadecimal #bcd #numeracao #conversao